



UNIVERSIDAD SERGIO ARBOLEDA

LENGUAJES DE PROGRAMACIÓN Y TRANSDUCCIÓN

JUAN VASQUEZ

SANTIAGO RODRIGUEZ

ROGER BOHORQUEZ

DOCENTE: WILLIAM FRASSER

BOGOTÁ D.C

2026

Contenido

INTRODUCCIÓN	3
OBJETIVO GENERAL	3
OBJETIVOS ESPECÍFICOS	4
ALFABETO.....	4
DESCRIPCIÓN DEL LENGUAJE	5
TIPO DE LENGUAJE	5
ESTRUCTURAS QUE SOPORTA	6
EXPRESIONES REGULARES.....	6
TOKENS DEL LENGUAJE.....	8
PALABRAS RESERVADAS.....	11
IMPLEMENTACIÓN.....	12
CASOS DE PRUEBA.....	12
• Caso 1: DECLARACIÓN DE VARIABLES	12
• Caso 2: ESTRUCTURA DE CONTROL (BUCLE FOR)	13
• Caso 3: NÚMEROS REALES.....	13
• Caso 4: MANEJO DE COMENTARIOS.....	14
• Caso 5: ERROR LÉXICO	14
RESULTADO DE CASOS	15
CONCLUSIONES	17
REFERENCIAS	17

INTRODUCCIÓN

Un compilador es básicamente un programa que puede leer un programa en un lenguaje, el lenguaje fuente, y traducirlo en un programa equivalente en otro lenguaje, el lenguaje destino (Aho et al., 2006). A la hora de desarrollar un compilador es fundamental transformar el código escrito en un lenguaje de programación de manera que pueda ser interpretado y procesado por la máquina. Por ello, es necesario un proceso que permita transformar dicho código en elementos comprensibles para la computadora: el análisis léxico.

El analizador léxico es la primera etapa de un compilador y se encarga de leer el código fuente y dividirlo en unidades llamadas tokens, los cuales pueden clasificarse como palabras reservadas, identificadores, operadores o constantes (Aho et al., 2006). El análisis permite también detectar posibles errores léxicos en el código, lo cual es fundamental pues constituye la base para las posteriores fases del compilador, asegurando que el código se interprete correctamente en niveles sintácticos y semánticos.

OBJETIVO GENERAL

El objetivo general de este trabajo es implementar un analizador léxico utilizando la herramienta JFlex, capaz de reconocer los diferentes tokens del lenguaje de programación “BASIC” a partir de un archivo de entrada.

Desarrollar un analizador léxico para el lenguaje BASIC utilizando la herramienta JFlex, capaz de identificar y clasificar los diferentes tokens presentes a partir de un archivo de entrada.

OBJETIVOS ESPECÍFICOS

1. Definir las reglas léxicas del lenguaje mediante expresiones regulares en un archivo .flex
2. Generar automáticamente el analizador léxico utilizando JFlex.
3. Implementar un programa en Java que permita leer un archivo de prueba y procesar los tokens generados.
4. Identificar y clasificar correctamente elementos como palabras reservadas, identificadores y operadores.
5. Detectar errores léxicos en el código fuente.
6. Mostrar los resultados del análisis de manera clara y comprensible.

ALFABETO

El alfabeto del lenguaje define el conjunto de símbolos válidos que pueden aparecer en el código fuente. Se incluyen letras, dígitos, operadores y símbolos básicos de agrupación.

$\Sigma = \{ a-z, A-Z, 0-9, +, -, *, /, \backslash, ^, =, <, >, \&, !, (,), ,, ;, :, ", \$, \%, \#, \text{espacio}, \text{tabulación}, \text{salto de línea} \}$

Incluye:

- Letras y dígitos para identificadores
- Operadores aritméticos y relacionales
- Delimitadores
- Comillas para cadenas
- Signos de tipo (sufijos de identificador)

- Símbolos adicionales propios como : y \$

DESCRIPCIÓN DEL LENGUAJE

BASIC es un lenguaje imperativo, de tipado débil y con una sintaxis orientada a líneas. Sus características léxicas más relevantes son:

- Insensibilidad a mayúsculas/minúsculas: IF, if e If son equivalentes. El escáner debe normalizar los lexemas.
- Sufijos de tipo en identificadores: una variable puede terminar con \$ (cadena), % (entero), ! (simple precisión), # (doble precisión) o & (long).
- Números de línea: en el dialecto clásico (GW-BASIC), cada sentencia comienza con un número de línea entero positivo.
- Comentarios: se inician con la palabra reservada REM o con el carácter apóstrofo (') y se extienden hasta el final de la línea.
- Operadores con doble rol: el signo = funciona como operador de asignación (LET x = 5) y como operador relacional (IF x = 5 THEN). El contexto sintáctico resuelve la ambigüedad.
- División entera: el operador \ realiza división entera, distinto de / que es división real.
- Concatenación: el operador & concatena cadenas (alternativo al + en algunos dialectos).

TIPO DE LENGUAJE

Según la Jerarquía de Chomsky, BASIC es un lenguaje de Tipo 2 (lenguaje libre de contexto) en su análisis sintáctico. Sin embargo, a nivel léxico, todos los tokens de BASIC son

definibles mediante gramáticas de Tipo 3 (regulares), lo que hace posible su reconocimiento mediante autómatas finitos deterministas (AFD) y, por tanto, su especificación mediante expresiones regulares.

ESTRUCTURAS QUE SOPORTA

- Declaración de variables con DIM y tipos explícitos (DIM x AS INTEGER).
- Asignación simple: LET x = expresión (o x = expresión sin LET).
- Condicional simple y compuesta: IF...THEN / IF...THEN...ELSE / IF...ELSEIF...ELSE...END IF.
- Selección múltiple: SELECT CASE...END SELECT.
- Bucle contado: FOR variable = inicio TO fin [STEP incremento] ... NEXT variable.
- Bucle con precondición: WHILE condición ... WEND.
- Bucle con postcondición: DO ... LOOP UNTIL condición / DO WHILE condición ... LOOP.
- Subrutinas y funciones: SUB nombre() ... END SUB / FUNCTION nombre() ... END FUNCTION.
- Entrada/Salida: PRINT, INPUT, LINE INPUT, OPEN, CLOSE, WRITE#, INPUT#.
- Saltos: GOTO etiqueta, GOSUB subrutina, RETURN.
- Arreglos: DIM arr(10) AS INTEGER, acceso arr(i).

EXPRESIONES REGULARES

Las expresiones regulares permiten definir los patrones para reconocer los diferentes tokens del lenguaje.

- LETRA = [a-zA-Z]
- DIGITO = [0-9]
- ALFANUM = [a-zA-Z0-9_]
- SIGNO_TIPO = [\$%! #&]
- ESPACIO = [\t]+
- FIN_LINEA = \n\r\n
- ID = LETRA (ALFANUM)* (SIGNO_TIPO)?
- ¿ENTERO = [+]? DIGITO+
- ¿REAL = [+]? DIGITO+ (\. ¿DIGITO+)? (¿[Ee] [+]? ¿DIGITO+)? | [+]? \. [0-9]+([Ee][+]?[0-9]+)?
- CADENA = "[^"]*"
- OP_ARIT = \+ | - | * | / | \\\ | \^ | MOD
- OP_REL = = | < | > | <= | >=
- OP_LOG = AND | OR | NOT
- OP_CONCAT = &
- PAREN_AB = \ (
- PAREN_CIE = \)
- COMA = ,
- PUNTO_COMA = ;
- DOS_PUNTOS = :
- COMENTARIO = REM[^\\n]* | '[^\\n]*
- EOL = \n
- NRO_LINEA = DIGITO+

TOKENS DEL LENGUAJE

A continuación, se presentan los tokens definidos para el lenguaje junto con su descripción y ejemplos.

TOKEN	DESCRIPCIÓN	EJEMPLO
LET	Asignación de valores	LET x = 10
PRINT	Salida de datos	PRINT "Hola"
INPUT	Entrada de datos	INPUT nombre\$
IF	Estructura condicional	IF x > 0 THEN
THEN	Parte de la condición	THEN
ELSE	Alternativa condicional	ELSE
FOR	Inicio de ciclo	FOR i = 1 TO 10

NEXT	Fin de ciclo FOR	NEXT i
WHILE	Inicio de ciclo	WHILE x < 10
WEND	Fin de ciclo WHILE	WEND
DIM	Declaración de variable	DIM x AS INTEGER
AS	Asignación de tipo	AS DOUBLE
INTEGER	Tipo de dato entero	INTEGER
DOUBLE	Tipo de dato real	DOUBLE
STRING	Tipo de dato texto	STRING
FUNCTION	Definición de función	FUNCTION suma(a, b)
SUB	Definición de subrutina	SUB mostrar()
END	Finalización de bloque	END IF

AND	Operador lógico	IF $x > 0$ AND $y > 0$
OR	Operador lógico	IF $x = 1$ OR $y = 2$
NOT	Operador lógico	IF NOT $x = 0$
ID	Identificador	x, total, nombre\$
ENTERO	Literal numérico entero	10, -5, +20
REAL	Literal numérico decimal	3.14, -0.25
CADENA	Literal de texto	"Hola mundo"
OP_ARIT	Operadores aritméticos	$x + y$, $a * b$, $x \setminus 2$
OP_REL	Operadores relacionales	$x = 5$, $x < 0$, $x \geq 1$
OP_CONCAT	Concatenación de cadenas	nombre\$ & apellido\$
SIMBOLO	Símbolos especiales	(), , ; :

COMENTARIO	Comentario de línea	REM esto es un comentario
EOL	Fin de línea	salto de línea

PALABRAS RESERVADAS

Las palabras reservadas de BASIC no pueden ser usadas como identificadores. Se organizan en las siguientes categorías:

CATEGORÍA	PALABRAS RESERVADAS
TIPOS DE DATOS	INTEGER, SINGLE, DOUBLE, STRING
DECLARACIÓN	DIM, AS
CONTROL DE FLUJO	IF, THEN, ELSE, END
BUCLES	FOR, TO, STEP, NEXT, WHILE, WEND
SUBROUTINAS	SUB, FUNCTION, RETURN
ENTRADA/SALIDA	PRINT, INPUT
MISCELÁNEO	LET, REM, STOP
OPERADORES LÓGICOS	AND, OR, NOT
FUNCIONES CADENA	LEN, LEFT\$, RIGHT\$, MID\$

IMPLEMENTACIÓN

Para el desarrollo del analizador léxico se utilizó JFlex, una herramienta que permite generar analizadores léxicos de forma automática a partir de reglas definidas mediante expresiones regulares. JFlex facilita la construcción del lexer al transformar un archivo .flex en código Java funcional.

El proyecto se organiza de la siguiente manera:

1. src/analizador/
 - Lexer.flex: contiene las reglas léxicas del lenguaje.
 - Lexer.java: archivo generado automáticamente por JFlex.
 - Tokens.java: define los tipos de tokens reconocidos.
 - FrontEnd.java: clase principal que ejecuta el análisis léxico.
 - Principal.java: permite generar el lexer desde el archivo .flex.
2. Pruebas/
 - prueba1.txt: archivo de entrada con el código a analizar.

CASOS DE PRUEBA

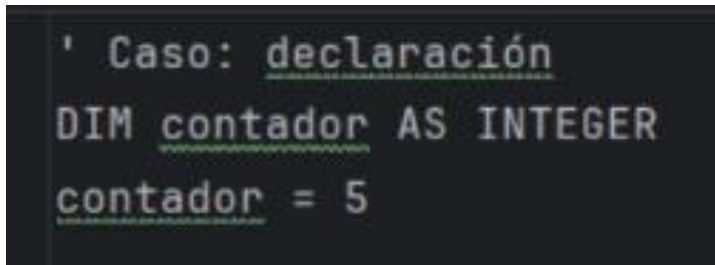
- **Caso 1: DECLARACIÓN DE VARIABLES**

Entrada:

- DIM contador AS INTEGER

- contador = 5

RESULTADO ESPERADO: Reconocimiento de palabras reservadas (DIM, AS, INTEGER), identificación de contador como identificador, reconocimiento de 5 como número.



```
' Caso: declaración
DIM contador AS INTEGER
contador = 5
```

- **Caso 2: ESTRUCTURA DE CONTROL (BUCLE FOR)**

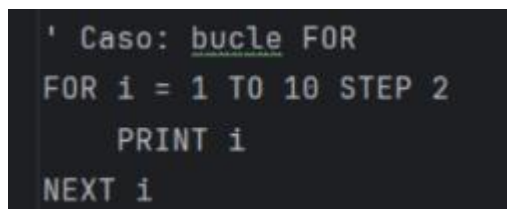
Entrada:

- FOR i = 1 TO 10 STEP 2

PRINT i

NEXT i

Resultado esperado: Reconocimiento de palabras reservadas (FOR, TO, STEP, PRINT, NEXT), identificación de i como identificador, reconocimiento de números (1, 10, 2).



```
' Caso: bucle FOR
FOR i = 1 TO 10 STEP 2
  PRINT i
NEXT i
```

- **Caso 3: NÚMEROS REALES**

Entrada:

- DIM precio AS DOUBLE

precio = 3.14

Resultado esperado: Reconocimiento de DOUBLE como tipo de dato, identificación de 3.14 como número real.

```
' Caso: número real
DIM precio AS DOUBLE
precio = 3.14
```

- **Caso 4: MANEJO DE COMENTARIOS**

Entrada:

- REM esto no debe generar tokens

Resultado esperado: Los comentarios deben ser ignorados (La sentencia REM permite crear un comentario), no deben generarse tokens en la salida.

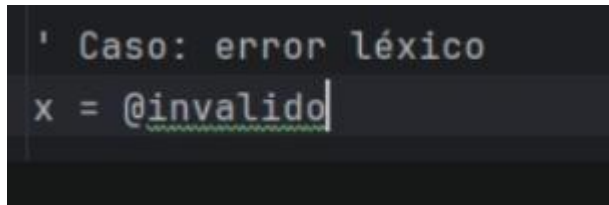
```
' Caso: comentario REM
REM esto no debe generar tokens
```

- **Caso 5: ERROR LÉXICO**

Entrada:

- x = @invalido

Resultado esperado: Identificación de x como identificador, reconocimiento de = como operador, detección de @ como símbolo no válido (error léxico), identificación de invalido como identificador



```
' Caso: error léxico  
x = @invalido|
```

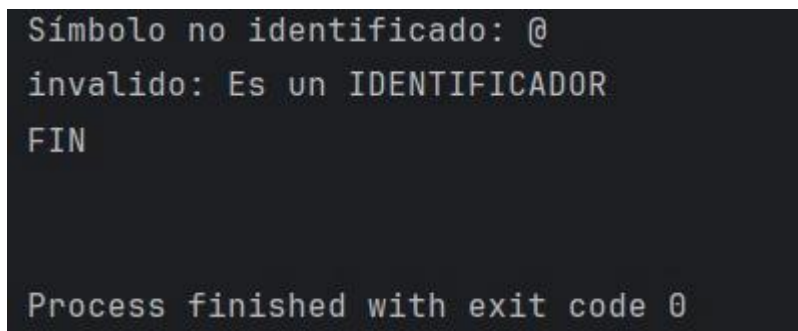
RESULTADO DE CASOS

Al ejecutar el analizador léxico con los casos de prueba definidos, se obtuvo una correcta identificación y clasificación de los diferentes elementos del lenguaje.

En cuanto a los valores numéricos, el analizador logró diferenciar tanto números enteros (5, 1, 10, 2) como números reales (3.14), clasificándolos de manera adecuada.

Los comentarios, tanto con ' como con REM, fueron ignorados correctamente, lo que indica que no interfieren en el análisis léxico del código.

El sistema también pudo identificar errores léxicos, como en el caso del símbolo @, el cual fue reportado como un carácter no válido dentro del lenguaje.



```
Símbolo no identificado: @  
invalido: Es un IDENTIFICADOR  
FIN  
  
Process finished with exit code 0
```

```
Token: DIM
contador: Es un IDENTIFICADOR
Token: AS
Token: INTEGER
contador: Es un IDENTIFICADOR
Token: OP_REL
5: Es un NUMERO
Token: FOR
i: Es un IDENTIFICADOR
Token: OP_REL
1: Es un NUMERO
Token: TO
10: Es un NUMERO
Token: STEP
2: Es un NUMERO
Token: PRINT
i: Es un IDENTIFICADOR
Token: NEXT
i: Es un IDENTIFICADOR
Token: DIM
precio: Es un IDENTIFICADOR
Token: AS
Token: DOUBLE
precio: Es un IDENTIFICADOR
Token: OP_REL
3.14: Es un NUMERO
x: Es un IDENTIFICADOR
Token: OP_REL
```


CONCLUSIONES

- El análisis léxico es fundamental porque constituye la primera fase del compilador. Permite transformar el código fuente en una secuencia de tokens, facilitando el trabajo de las siguientes etapas como el análisis sintáctico y semántico.
- Se comprendió el funcionamiento de un analizador léxico, incluyendo la identificación de tokens como palabras reservadas, identificadores y números. Se aprendió a utilizar herramientas como JFlex para automatizar este proceso.
- Se presentaron dificultades al definir correctamente las expresiones regulares y manejar símbolos no reconocidos.

REFERENCIAS

- Compiladores: Principios, técnicas y herramientas. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson.